

DSA 期末考前学案：从模板到实例的迁移

结合错题 PDF、tree.md 与 graph.md 的详细复盘

根据当前文件夹资料整理

2026 年 6 月

目录

使用说明	3
1 作业体现的知识点与个人薄弱点	3
1.1 总体知识地图	3
1.2 最明显的个人薄弱点	3
1.2.1 薄弱点一：模板能读懂，但题目一变就不知道状态该怎么改	3
1.2.2 薄弱点二：容易把“最短路、路线数、最小生成树”混在一起	4
1.2.3 薄弱点三：树题容易过度建模	4
1.2.4 薄弱点四：输入输出口径容易被忽略	4
2 从 md 模板到实例代码的迁移方法	4
2.1 BFS 模板迁移：先问四个问题	4
2.1.1 普通 BFS	5
2.1.2 状态 BFS	5
2.1.3 最优值 BFS	5
2.1.4 Dijkstra：当 BFS 不再成立	6
2.2 Dijkstra 模板迁移：不只是“从起点到终点”	6
2.2.1 网格 Dijkstra	6
2.2.2 反向 Dijkstra + DP	6
2.3 Floyd 模板迁移：距离矩阵之外还要路径矩阵	7
2.4 MST 模板迁移：Prim 和 Kruskal 的选择	7
2.5 树模板迁移：输入形式决定递归方式	8
3 逐题详细复盘：作业与 md 的拓展关系	8
3.1 22 根据二叉树前中序序列建树	8
3.2 23 括号嵌套二叉树	9
3.3 24 二叉树	9
3.4 25 二叉搜索树的层次遍历	10
3.5 26 实现堆结构	10
3.6 27 树的转换	10
3.7 28 Huffman 编码树	11
3.8 49 二叉树的深度	11
3.9 50 扩展二叉树	11
3.10 53 根据二叉树中后序序列建树	11
3.11 54 验证二叉搜索树	12
3.12 1 单词序列	12

3.13	2 仙岛求药	12
3.14	3 变换的迷宫	12
3.15	1 拓扑排序	13
3.16	2 丛林中的路	13
3.17	3 A Walk Through the Forest	13
3.18	4 打怪救公主	13
3.19	5 兔子与樱花	14
3.20	56 拯救行动	14
3.21	57 连接所有点的最小费用	14
4	考试时的算法选择决策表	15
5	代码落地前的详细检查清单	15
6	考前重点训练顺序	15
6.1	第一优先级：模板变形题	15
6.2	第二优先级：树输入解析题	16
6.3	第三优先级：MST 与堆	16
7	最后的压缩记忆	16

使用说明

这份学案不是把 tree.md 和 graph.md 再抄一遍，而是围绕一个真实考试问题展开：模板代码能看懂，但换成题目实例之后，不知道该改哪里。复习时建议按如下顺序使用：

1. 先看第 1 节的总览，明确作业暴露出的薄弱点。
2. 再看第 2 节的“模板迁移法”，尤其是 BFS、Dijkstra、MST 和树递归。
3. 最后逐题看第 3 节，训练“看到题面如何把 md 里的模板改成能 AC 的代码”。

本节核心

这份资料的中心结论是：你不是不会模板，而是需要把模板拆成四个可替换部件：状态、转移、判重、答案口径。只要这四件事判断对，大多数题都能从已有模板自然改出来。

1 作业体现的知识点与个人薄弱点

1.1 总体知识地图

本次 PDF 作业一共覆盖 21 道题，可以分为两大块：树结构题与图搜索题。树题偏向“输入形式如何转成遍历/递归”，图题偏向“普通 BFS 什么时候需要升级或改状态”。

模块	涉及题目	真正考点
二叉树遍历与重建	22、23、49、50、53	前序/中序/后序之间的递归关系；扩展先序和括号嵌套输入的解析；空树出口如何设定。
BST 与完全二叉树编号	24、25、54	BST 插入与验证；重复值处理；完全二叉树编号不建树，用区间计算。
堆与 Huffman	26、28	heapq 的小顶堆性质；每组初始化；Huffman 的答案是每次合并代价之和。
BFS 与状态 BFS	单词序列、仙岛求药、变换的迷宫、打怪救公主	是否普通无权；状态是否要包含时间、资源、剩余血量；判重是否还能用布尔数组。
Dijkstra 与路径计数	拯救行动、A Walk Through the Forest	边权不同则不能普通 BFS；最短路结果可以作为后续 DP 的评价函数。
Floyd 与路径恢复	兔子与樱花	多源多查询；不仅要最短距离，还要恢复完整路径并按格式输出。
MST 与拓扑	拓扑排序、丛林中的路、连接所有点的最小费用	DAG 入度维护；编号小优先用最小堆；显式边用 Kruskal，完全图用 Prim。

1.2 最明显的个人薄弱点

1.2.1 薄弱点一：模板能读懂，但题目一变就不知道状态该怎么改

graph.md 里的 BFS 模板是标准的：

```
q = deque([(sx, sy, dist)])
visited[sx][sy] = True
while q:
    x, y, d = q.popleft()
    for dx, dy in dirs:
        ...
```

但是作业中真正容易错的地方是：状态不一定只有 (x, y)。例如：

- 变换的迷宫：同一个格子在不同时间模数下可走性不同，所以状态是 (x, y, t % K)。

- 打怪救公主：同一个格子如果以更多血量到达，后续更有希望，所以不能简单 visited，要用 `best[x][y]`。
- 拯救行动：进入不同格子的代价不同，普通队列按层扩展失效，要换成优先队列。

1.2.2 薄弱点二：容易把“最短路、路线数、最小生成树”混在一起

这三类题长得都像图题，但目标函数完全不同：

- 最短路：问从 A 到 B 或从某点到所有点的最小距离。
- 路线数：问满足某种合法条件的路径有多少条，通常要 DFS/DP 计数。
- 最小生成树：问把所有点连通且总代价最小，不关心从某点到某点的最短距离。

作业中的典型对比：

- 拯救行动是最短路，答案是最短时间。
- A Walk Through the Forest 先算最短路，但最终答案是合法路线数量。
- 丛林中的路和连接所有点的最小费用都是 MST，不是 Dijkstra。

1.2.3 薄弱点三：树题容易过度建模

`tree.md` 中很多代码使用节点类，这有助于理解。但作业里很多题并不要求真的建树：

- 22 前序 + 中序输出后序：可以直接递归字符串返回后序。
- 24 完全二叉树编号：完全不用树节点，只需每层编号区间。
- 28 Huffman WPL：只需要堆和累计代价，不必真的构造编码树。

考试中要先问自己：题目要输出结构，还是只要一个序列/数字？如果只要序列或数字，常常可以用更轻的实现。

1.2.4 薄弱点四：输入输出口径容易被忽略

很多错题不是算法本身错，而是口径错：

- 起点距离是 0 还是 1：问“时间”通常从 0；问“经过格子数”可能从 1。
- 多组输入如何结束：EOF、单独 0、0 0、先给 T，四种都出现过。
- 图是否无向：丛林中的路、兔子与樱花、A Walk Through the Forest 都要双向加边。
- 输出格式：拓扑排序是否带 `v` 前缀，兔子与樱花是否要 `A->(d)->B`。

2 从 md 模板到实例代码的迁移方法

2.1 BFS 模板迁移：先问四个问题

迁移方法

把任何 BFS 题都拆成四个问题：

1. 状态是什么？
2. 从一个状态能转移到哪些状态？
3. 什么情况下两个状态算“已经访问过”？
4. 答案是第一次到达、最短距离、最大资源，还是路径数量？

2.1.1 普通 BFS

适用条件：每条边代价相同，第一次到达某状态就是最优。

```
from collections import deque

q = deque([(sx, sy, 0)])
visited = [[False] * C for _ in range(R)]
visited[sx][sy] = True

while q:
    x, y, d = q.popleft()
    if (x, y) == (tx, ty):
        print(d)
        break
    for dx, dy in dirs:
        nx, ny = x + dx, y + dy
        if not in_bound(nx, ny):
            continue
        if wall(nx, ny) or visited[nx][ny]:
            continue
        visited[nx][ny] = True
        q.append((nx, ny, d + 1))
```

作业对应：仙岛求药、单词序列的 BFS 部分。注意单词序列里状态不是坐标，而是单词；仙岛求药里答案可能是经过格子数，起点距离可能设为 1。

2.1.2 状态 BFS

当同一个位置在不同附加状态下有不同意义时，visited 必须升维。

```
visited = [[[False] * K for _ in range(C)] for _ in range(R)]
q = deque([(sx, sy, 0)])
visited[sx][sy][0] = True

while q:
    x, y, t = q.popleft()
    nt = t + 1
    key = nt % K
    ...
    if not visited[nx][ny][key]:
        visited[nx][ny][key] = True
        q.append((nx, ny, nt))
```

作业对应：变换的迷宫。它的关键不是“迷宫 BFS”，而是“墙随时间变化，所以同一格不同时间模数是不同状态”。

2.1.3 最优值 BFS

当题目目标不是最短步数，而是“剩余资源最大”时，布尔 visited 会误杀好状态。

```
best = [[-1] * C for _ in range(R)]
best[sx][sy] = initial_hp
q = deque([(sx, sy)])

while q:
    x, y = q.popleft()
    cur = best[x][y]
    for dx, dy in dirs:
        nx, ny = x + dx, y + dy
        nhp = cur - cost(nx, ny)
        if nhp <= 0:
            continue
        if nhp > best[nx][ny]:
            best[nx][ny] = nhp
            q.append((nx, ny))
```

作业对应：打怪救公主。你要把“是否访问过”改成“是否以更好的血量访问过”。

2.1.4 Dijkstra: 当 BFS 不再成立

只要进入不同邻居的代价不同，普通 BFS 的“按层最短”就失效。

```
import heapq

dist = [[float('inf')] * C for _ in range(R)]
dist[sx][sy] = 0
pq = [(0, sx, sy)]

while pq:
    d, x, y = heapq.heappop(pq)
    if d != dist[x][y]:
        continue
    if (x, y) == (tx, ty):
        print(d)
        break
    for dx, dy in dirs:
        nx, ny = x + dx, y + dy
        w = cost(nx, ny)
        nd = d + w
        if nd < dist[nx][ny]:
            dist[nx][ny] = nd
            heapq.heappush(pq, (nd, nx, ny))
```

作业对应：拯救行动。普通格代价 1，守卫格代价 2，所以要用优先队列。

2.2 Dijkstra 模板迁移：不只是“从起点到终点”

在 graph.md 里，Dijkstra 是从一个起点到所有点。作业中有两个重要变形。

2.2.1 网格 Dijkstra

拯救行动把网格看作图：

- 每个格子是一个顶点。
- 上下左右相邻是边。
- 进入 x 的代价是 2，进入普通格/终点的代价是 1。

所以 Dijkstra 的图不一定要显式存邻接表，可以在扩展当前格子时动态枚举四个方向。

2.2.2 反向 Dijkstra + DP

A Walk Through the Forest 的关键是“每一步更接近家”。“更接近家”不是边权更小，而是到家的最短距离更小。于是：

1. 从家 2 出发跑 Dijkstra，得到每个点到家的最短距离 $\text{dist}[u]$ 。
2. 从办公室 1 开始 DFS。
3. 只能走向满足 $\text{dist}[v] < \text{dist}[u]$ 的邻居。
4. 用 memo 记忆化每个点到家的合法路线数。

```
def count(u):
    if u == 2:
        return 1
    if memo[u] != -1:
        return memo[u]
    ans = 0
    for v, w in graph[u]:
        if dist[v] < dist[u]:
            ans += count(v)
    memo[u] = ans
    return ans
```

高频失误

这题不是求最短路径条数。只要每一步到家的最短距离严格下降，路径就合法，即使这条路径本身不是从 1 到 2 的最短路径。

2.3 Floyd 模板迁移：距离矩阵之外还要路径矩阵

兔子与樱花是典型的“点数小 + 多次查询 + 要输出路径”的题。只用 Dijkstra 也可以，但每次查询都跑会麻烦；Floyd 一次预处理全部点对更自然。

```
dist = [[INF] * n for _ in range(n)]
nxt = [[-1] * n for _ in range(n)]
for i in range(n):
    dist[i][i] = 0
    nxt[i][i] = i

for u, v, w in edges:
    if w < dist[u][v]:
        dist[u][v] = dist[v][u] = w
        nxt[u][v] = v
        nxt[v][u] = u

for k in range(n):
    for i in range(n):
        for j in range(n):
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]
                nxt[i][j] = nxt[i][k]
```

路径恢复：

```
def get_path(u, v):
    if nxt[u][v] == -1:
        return []
    path = [u]
    while u != v:
        u = nxt[u][v]
        path.append(u)
    return path
```

2.4 MST 模板迁移：Prim 和 Kruskal 的选择

场景	优先算法	原因
边已经显式给出， 例如 A B 12	Kruskal	直接收集边、排序、并查集判断是否成环。
任意两点都能连边， 边权由坐标计算	Prim	不必生成所有边；维护每个未选点到已选集合的最小连接代价。
题目强调“连通且总 代价最小”	MST	不要误用 Dijkstra；Dijkstra 是从某点出发的最短路。

Kruskal 核心：

```
edges.sort()
parent = list(range(n))

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(a, b):
```

```
ra, rb = find(a), find(b)
if ra == rb:
    return False
parent[rb] = ra
return True

ans = 0
cnt = 0
for w, u, v in edges:
    if union(u, v):
        ans += w
        cnt += 1
        if cnt == n - 1:
            break
```

Prim 核心:

```
used = [False] * n
minDist = [float('inf')] * n
minDist[0] = 0
ans = 0

for _ in range(n):
    u = -1
    for i in range(n):
        if not used[i] and (u == -1 or minDist[i] < minDist[u]):
            u = i
    used[u] = True
    ans += minDist[u]

    for v in range(n):
        if not used[v]:
            w = weight(u, v)
            if w < minDist[v]:
                minDist[v] = w
```

2.5 树模板迁移：输入形式决定递归方式

输入形式	根的位置	切分方法
前序 + 中序	前序第一个	在中序找根；左子树长度决定前序切片。
中序 + 后序	后序最后一个	在中序找根；左子树长度决定后序切片。
扩展先序	当前字符	当前字符是 . 则为空；否则先建左再建右。
括号嵌套	子串第一个字符	括号内部按最外层逗号切左右子树。
编号左右孩子	根通常由题面给出	用数组保存左右孩子，递归传编号。

3 逐题详细复盘：作业与 md 的拓展关系

3.1 22 根据二叉树前中序序列建树

本节核心

md 来源：tree.md 中“重建二叉树”的前序 + 中序原则。作业拓展：多组输入到 EOF，输出后序。

模板迁移：

- 1. md 中说明 pre[0] 是根。

2. 在中序中找到根的位置 k 。
3. 中序左半边长度就是左子树节点数。
4. 前序切片不能随便切，必须按左子树长度切。
5. 输出后序，所以递归结果是 $\text{left} + \text{right} + \text{root}$ 。

高频失误

本题最容易错在前序切片：左子树前序不是 $\text{pre}[1:k]$ ，而是 $\text{pre}[1:1+\text{len}(\text{left_in})]$ 。因为 k 是根在中序中的位置，不是前序中的位置。

可背思路：

```
def post(pre, ino):
    if not pre:
        return ''
    root = pre[0]
    k = ino.index(root)
    left_in = ino[:k]
    right_in = ino[k+1:]
    left_pre = pre[1:1+len(left_in)]
    right_pre = pre[1+len(left_in):]
    return post(left_pre, left_in) + post(right_pre, right_in) + root
```

3.2 23 括号嵌套二叉树

md 中主要是普通树和二叉树遍历，本题补充的是“字符串解析”。如果直接 `split(',')`，遇到 $A(B(*,C),D(E))$ 会把内部逗号也切掉。

迁移方法：

1. 当前子串为空或 $*$ ，返回空树。
2. 当前子串第一个字符是根。
3. 如果长度为 1，就是叶子。
4. 去掉外层根和括号，得到内部字符串。
5. 扫描内部字符串，维护括号深度；只在深度为 0 的逗号处分割。

核心代码：

```
def split_children(s):
    depth = 0
    for i, ch in enumerate(s):
        if ch == '(':
            depth += 1
        elif ch == ')':
            depth -= 1
        elif ch == ',' and depth == 0:
            return s[:i], s[i+1:]
    return s, ''
```

这题是“树 + 解析”的组合题。复习时不要只背遍历，还要练习如何把题面给的字符串表示转成左右子树。

3.3 24 二叉树

本节核心

md 来源：完全二叉树的编号性质：节点 p 的左孩子是 $2p$ ，右孩子是 $2p+1$ 。作业拓展：求编号 m 所在子树实际存在的节点数。

不要建树。因为 n 可很大，建树既慢又浪费。以 m 为根的子树每一层编号都是连续区间：

```

第0层: [m, m]
第1层: [2m, 2m+1]
第2层: [4m, 4m+3]
下一层: left *= 2, right = right * 2 + 1

```

每层贡献:

```
ans += min(right, n) - left + 1
```

前提是 $\text{left} \leq n$ ，否则这一层已经没有实际存在的节点。

3.4 25 二叉搜索树的层次遍历

md 中有 BST 插入、树节点、层序 BFS。作业把它们拼起来，额外强调重复值只插入一次。

迁移方法:

1. 输入一串数字。
2. 按顺序插入 BST。
3. 插入时小于走左，大于走右，等于直接忽略。
4. 建好树后用 deque 层序遍历。
5. 输出时用 ' '.join(ans)，避免末尾空格。

高频错误:

- 把重复值插入左边或右边，导致层序结果多数字。
- 用 `list.pop(0)` 虽然能跑小数据，但 `deque.popleft()` 更规范。
- 空输入时访问空根，不过本题通常保证有输入。

3.5 26 实现堆结构

md 中既有手写二叉堆，也有 Python `heapq`。作业更适合直接用 `heapq`。

关键迁移:

- 题目要求“输出并删除最小元素”，而 `heapq` 默认就是最小堆。
- 每一组测试数据都要重新 `heap=[]`。
- `type=1` 后面有一个数，`type=2` 没有额外参数。

API 检查:

```

import heapq
heapq.heappush(heap, x)
x = heapq.heappop(heap)
heapq.heapify(arr)

```

3.6 27 树的转换

md 中讲“左儿子右兄弟”转换：第一个孩子变左儿子，其余孩子串成右兄弟。作业问高度，不要求输出树。

理解重点:

- 原树高度：只看父子关系向下走了多少层。
- 转换后二叉树高度：左孩子边和右兄弟边都算二叉树的边。
- 所以兄弟多时，转换后二叉树高度可能显著增加。

本题真正考的是“能否只维护答案，不真的构造结构”。看到只求高度，可以用栈模拟 DFS 过程，把当前节点对应的二叉深度存在栈里。

3.7 28 Huffman 编码树

md 中讲了完整 Huffman 树构造和编码，作业只要求最小 WPL。

最小 WPL 的核心解释：

- 每次合并两个最小权值 a 和 b 。
- 这次合并会让这两个子树里的所有叶子深度加 1。
- 增加的 WPL 正好是 $a+b$ 。
- 所以答案累计所有合并值。

不要把答案写成最后堆中根权值。最后根权值只是所有叶子权值总和，不是 WPL。

3.8 49 二叉树的深度

md 中的深度递归是 $\max(\text{left}, \text{right})+1$ 。作业输入不是节点对象，而是编号描述：

- 第 i 行给节点 i 的左右孩子。
- -1 表示空。
- 根节点明确是 1。

所以把 $\text{depth}(\text{root})$ 改成 $\text{depth}(i)$ ：

```
def depth(i):
    if i == -1:
        return 0
    return max(depth(left[i]), depth(right[i])) + 1
```

3.9 50 扩展二叉树

扩展二叉树输入是先序序列， $.$ 表示空节点。它和普通先序的区别是：普通先序无法唯一确定树，但带空标记的先序可以。

迁移步骤：

1. 用全局下标 pos 指向当前读到的字符。
2. 读一个字符后 $\text{pos} += 1$ 。
3. 如果字符是 $.$ ，返回空。
4. 否则创建节点，并递归构建左子树、右子树。
5. 最后输出中序和后序，遍历时忽略空节点。

3.10 53 根据二叉树中后序序列建树

它和第 22 题是镜像关系。前中题根在前序开头；中后题根在后序末尾。

可背思路：

```
def preorder(ino, post):
    if not ino:
        return ''
    root = post[-1]
    k = ino.index(root)
    left_in = ino[:k]
    right_in = ino[k+1:]
    left_post = post[:len(left_in)]
    right_post = post[len(left_in):-1]
    return root + preorder(left_in, left_post) + preorder(right_in, right_post)
```

高频错误：后序切右子树时不要包含最后的根。

3.11 54 验证二叉搜索树

md 中讲 BST 插入和查找，作业问“给定树是否是 BST”。判断 BST 合法不能只看父子。

反例：

```

10
 / \
5   15
 /
6

```

节点 6 小于 15，看起来是 15 的合法左孩子；但它在 10 的右子树里，必须大于 10，所以整棵树不是 BST。

正确方法是上下界递归：

```

def valid(node, low, high):
    if node is None:
        return True
    if not (low < node.val < high):
        return False
    return valid(node.left, low, node.val) and valid(node.right, node.val, high)

```

3.12 1 单词序列

graph.md 的词梯问题就是本题基础。迁移时注意三件事：

1. 点是单词，不是数字编号。
2. 两个单词只差一个字母时有边。
3. 如果输出的是变换序列长度，起点长度通常是 1，不是 0。

建图可以两种方式：

- 暴力比较每两个单词，简单但可能慢。
- 桶方法：把每个位置替换成 _，同桶单词互相可达。

3.13 2 仙岛求药

这是普通网格 BFS，但要特别注意答案口径。

迁移检查：

- 地图中的起点和终点字符是什么？
- 墙是什么字符？
- 输出的是移动时间，还是经过格子数？
- 不可达输出什么？

如果题目说“经过的方格数包括起点”，起点距离设为 1。如果题目说“花费时间”，起点时间设为 0。

3.14 3 变换的迷宫

这是状态 BFS 的重点题。普通 `visited[x][y]` 会错，因为同一个格子在不同时间可能可走性不同。

迁移方法：

1. 状态写成 (x, y, t) 。
2. 判重写成 `visited[x][y][t % K]`。
3. 每走一步，时间 $t+1$ 。
4. 判断下一格是否可走时，要用下一时刻的时间模数。

记忆句：只要题目出现周期、时间变化、白天黑夜、机关开关，就先想 `time % period`。

3.15 1 拓扑排序

graph.md 中拓扑排序用普通队列即可。但作业里常见额外要求：同等条件下编号小的点先输出。

迁移：

- 如果只要求任意拓扑序：用 deque。
- 如果要求编号小优先：用 heapq 存入度为 0 的点。
- 如果最后输出点数不足 n：说明有环。

核心逻辑：

```
for v in graph[u]:
    indeg[v] -= 1
    if indeg[v] == 0:
        heapq.heappush(heap, v)
```

3.16 2 丛林中的路

这是 Kruskal MST。md 里讲的并查集在这里完整使用。

作业特殊点：

- 顶点是大写字母，需要转成编号：ord(ch)-ord('A')。
- 输入每行可能包含多个邻接边，要循环读。
- 是无向图，但 Kruskal 只需要边集合，不必存双向邻接表。

3.17 3 A Walk Through the Forest

本题是组合题，非常适合作为考前重点。

题面含义：

- 家是 2，办公室是 1。
- Jimmy 每一步必须更接近家。
- “更接近”指到家 2 的最短距离更小。

算法组合：

1. 建无向带权图。
2. 从 2 跑 Dijkstra，得到 dist[u]。
3. 从 1 做 DFS 计数，只能走向 dist[v] < dist[u] 的点。
4. 用 memo[u] 避免重复计算。

高频失误

这题最容易误解成“求从 1 到 2 的最短路径条数”。实际上，只要每一步越来越接近家，整条路径就合法，它可能不是最短路径。

3.18 4 打怪救公主

本题问“到达公主时最大剩余血量”，所以不是普通最短路径题。

模板改法：

- 普通 BFS 的 visited[x][y] 改成 best[x][y]。
- best[x][y] 表示目前已知到达该格子的最大剩余血量。
- 如果新路径剩余血量更大，即使这个格子以前来过，也要重新入队。

判断：

- 进入数字格扣血。
- 起点、终点、普通路不扣血。
- 血量扣到 0 或以下不能继续。

3.19 5 兔子与樱花

这是 Floyd + 路径恢复 + 字符串映射。

完整迁移：

1. 读地点名，建立 `id_of[name]`。
2. 初始化 `dist` 和 `nxt`。
3. 读无向边，更新两边。
4. Floyd 三重循环更新最短路。
5. 查询时恢复点序列。
6. 按 `A->(d)->B` 输出每一段。

注意：输出每段的 `d` 是路径上相邻两点的边长，不是从起点到该点的累计距离。

3.20 56 拯救行动

这是网格 Dijkstra。它和打怪救公主看起来都像“救人迷宫”，但目标不同：

- 打怪救公主：最大剩余血量。
- 拯救行动：最短时间。

所以拯救行动要用 `dist[x][y]`，表示到达该格子的最短时间。

代价：

- 进入 @、r、a：代价 1。
- 进入 x：代价 2。
- 进入 #：不允许。

3.21 57 连接所有点的最小费用

这是完全图上的 MST。

题面信号：

- “连接所有点”
- “总费用最小”
- “任意两点之间有且仅有一条简单路径”

这三个信号都指向最小生成树。

为什么用 Prim：

- 任意两点都有边，是完全图。
- 如果建所有边再排序，需要 $O(n^2)$ 条边。
- Prim 可以边算曼哈顿距离边更新 `minDist`。

4 考试时的算法选择决策表

题面信号	选择	理由与改法
每一步代价相同，问最少步数	BFS	队列按层扩展，第一次到达终点最优。
地图/规则随时间周期变化	状态 BFS	状态加入 $t\%K$ ，visited 升维。
同一位置带不同资源	最优值 BFS 或状态 BFS	资源影响未来，布尔 visited 不够。
进入不同点代价不同	Dijkstra	普通 BFS 按步数，不按总代价。
多次询问任意两点最短路	Floyd	点数小，用三重循环一次预处理。
要求输出最短路径本身	BFS parent / Floyd nxt / Dijkstra pred	只存距离不够，要保存路径恢复信息。
所有点连通且总代价最小	MST	与从某点出发最短路无关。
显式给边的 MST	Kruskal	排序边 + 并查集。
完全图坐标 MST	Prim	不建全边，动态算边权。
有向依赖顺序	拓扑排序	入度为 0 的点逐步输出。
编号小优先的拓扑序	堆优化拓扑	入度 0 的集合用最小堆。
前序 + 中序	树递归	根在前序开头，中序切左右。
中序 + 后序	树递归	根在后序末尾，中序切左右。
Huffman / 最优二叉树 / WPL	最小堆	每次合并两个最小，累计合并值。

5 代码落地前的详细检查清单

考前检查

1. 输入规模是否允许建图/建树？如果 n 很大，避免显式建树或建完全图边集。
2. 输入结束条件是什么？EOF、0、0 0、还是第一行 T。
3. 起点答案初始化是 0 还是 1？看题目问“时间/步数”还是“经过格子数/序列长度”。
4. 图是有向还是无向？无向图读边时两边都要加，Kruskal 边集则只存一份即可。
5. 点编号从 0 还是 1 开始？若题目是 1 到 n ，数组一般开 $n+1$ 。
6. visited 是布尔吗？如果题目有时间、钥匙、血量、剩余次数，通常不是简单二维布尔。
7. BFS 是否仍然成立？只要边权不是全相等，就考虑 Dijkstra。
8. 是否要输出路径？要输出路径就要保存 parent/nxt/pred。
9. 是否要求字典序或编号小优先？队列可能要换成堆。
10. 输出格式有没有箭头、括号、前缀、空格、大小写固定要求？

6 考前重点训练顺序

6.1 第一优先级：模板变形题

1. 变换的迷宫：训练状态 BFS 和 `visited[x][y][t%K]`。
2. 拯救行动：训练普通 BFS 升级 Dijkstra。
3. 打怪救公主：训练布尔 visited 改最优值数组。

4. A Walk Through the Forest: 训练 Dijkstra + 记忆化 DFS 计数。
5. 兔子与樱花: 训练 Floyd + 路径恢复 + 字符串映射。

6.2 第二优先级: 树输入解析题

1. 22 与 53 成对复习, 区分根在前序开头还是后序末尾。
2. 23 单独复习括号嵌套, 重点是最外层逗号。
3. 50 单独复习扩展先序, 重点是 . 的空节点出口。
4. 54 单独复习 BST 验证, 重点是上下界。

6.3 第三优先级: MST 与堆

1. 丛林中的路: 显式边 Kruskal。
2. 连接所有点的最小费用: 完全图 Prim。
3. 实现堆结构: heapq 操作流。
4. Huffman 编码树: 堆合并与 WPL。

7 最后的压缩记忆

本节核心

- BFS 不是万能的; 它只保证“边权相同”时第一次到达最优。
- 状态不止位置; 时间、血量、钥匙、剩余次数都可能是状态的一部分。
- 最短路不是 MST; “连通所有点总代价最小”才是 MST。
- 树题先看输入形式, 再决定递归根在哪里。
- 题目只问数字或序列时, 不必执着于完整建模。
- 输出格式和输入结束条件是最容易白丢分的地方。